

Reliable and efficient integration of AI into camera traps for smart wildlife monitoring based on continual learning

Delia Velasco-Montero^a, Jorge Fernández-Berni^{a,*}, Ricardo Carmona-Galán^a,
Ariadna Sanglas^b, Francisco Palomares^b

^a Instituto de Microelectrónica de Sevilla, CI Américo Vespucio, 28, Seville 41092, Spain

^b Estación Biológica de Doñana, CI Américo Vespucio, 26, Seville 41092, Spain

ARTICLE INFO

Keywords:

On-device processing
Wildlife monitoring
Smart camera trap
Hardware-software co-design
Efficient AI
Continual learning

ABSTRACT

In this paper, we comprehensively report on an efficient approach for the integration of artificial intelligence (AI) processing pipelines in camera traps for smart on-site wildlife monitoring. Our work covers hardware, software, and algorithmics. We have built two prototypes of smart camera trap on a maximum bill of materials of 100\$. We have also built two datasets, made publicly available, comprising over 17 k images, many of them notably challenging even for humans. Leveraging our broad expertise on embedded systems, specialized software libraries and toolchains, and AI techniques such as transfer learning, explainable AI, and, most importantly, continual learning, we achieve more reliable inference on-site - specifically 10 % higher F1-score - than Mega-Detector run off-site on a desktop computer. The paper includes many practical details on system realization and on-site training in addition to a vast set of lab and experimental results.

1. Introduction and related work

Camera traps, which can automatically record pictures and/or videos upon (ideally) activation by animal motion, constitute a mature commercial technology that allows keeping track of ecosystem variables such as species diversity, abundance, and distribution. These devices typically register a massive volume of visual data that requires off-site analysis. Fortunately, artificial intelligence (AI)-based automated software solutions have emerged in the last few years as powerful desktop tools to facilitate this analysis. In particular, they assist in tasks such as individual identification, species classification, and filtering of ‘blank’ images – i.e., images containing no animal.

A natural step forward in technological terms with respect to this scenario of off-site processing is automatic on-site analytics. A major advantage of this approach is that the above-mentioned massive volume of data would be sifted on the fly; thus, camera traps would only register meaningful information according to the deployment goal. Furthermore, there are specific application scenarios in which on-site analysis would provide a notable boost to the intended functionality. For instance, when it comes to capturing animals in the field, individuals belonging to species other than the targeted one may be unintentionally trapped. A smart camera trap integrated with the capture system may

automatically release those individuals after classifying them as species of no interest. Another example is the deployment of camera traps in wildland-urban interfaces to detect the presence of species potentially hazardous for humans or livestock and issue an early alarm.

Our research is focused on the realization of camera traps that incorporate reliable inference capabilities while keeping the same form factor, cost, and battery lifetime than commercial camera traps. This is a remarkable challenge for two fundamental reasons addressed in this paper: 1) mainstream AI algorithms demand a lot of computational and memory resources to reach the accuracy levels required by practical applications; 2) camera traps are to be placed at any location, with no control at all on the operation conditions and surrounding environment.

The manuscript is organized as follows. In [Section 1.1](#), we review related work to establish the scope of our contribution to the state of the art. In [Section 2](#), we provide an overview of the proposed methodology for implementation of smart camera traps. This methodology is further elaborated in [Section 3](#) (network training) and 4 (hardware-software integration), where a comparison with state-of-the-art smart camera traps is also provided. In [Section 5](#), we report field tests, propose algorithm refinements to improve performance, further elaborate on some system aspects, and provide a performance comparison – in terms of accuracy metrics – between the proposed methodology and related

* Corresponding author.

E-mail address: jfernandez8@us.es (J. Fernández-Berni).

<https://doi.org/10.1016/j.ecoinf.2024.102815>

Received 1 March 2024; Received in revised form 3 September 2024; Accepted 3 September 2024

Available online 8 September 2024

1574-9541/© 2024 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

approaches. Finally, Section 6 summarizes the challenges encountered and establishes guidelines for future work.

1.1. Related work

The emergence of AI in the discipline of biodiversity conservation is supported, among other key aspects, by the availability of large *datasets* to train machine learning, and more specifically deep learning (DL), models. These datasets are fed by all types of sensors deployed in the field: biologists for biometrics tracking, autonomous recording units for acoustic monitoring, camera traps for visual recognition, etc. LILA BC (*Labeled Information Library of Alexandria*, 2024) is a prominent example of *public repository* containing a variety of datasets related to biology and conservation. In particular, concerning the topic of this paper, it offers popular camera-trap datasets such as Snapshot Serengeti (SS) (L. I. L. of Alexandria, n.d.), Caltech Camera Traps (*Labeled Information Library of Alexandria: Biology and Conservation* (LILA BC), 2024a), and North American Camera Trap Images (*Labeled Information Library of Alexandria: Biology and Conservation* (LILA BC), 2024b). To the best of our knowledge, SS presently constitutes the largest collection, comprising 1.2 million capture events from 225 camera traps. Experts have labeled these images into 48 different species categories and have annotated animal behaviors and attributes – standing, resting, moving, eating, interacting, offspring presence, etc. Interestingly, the number of publicly accessible datasets is continuously growing thanks to the promotion of different projects and initiatives, e.g., SystemaNaturae (*Systema Naturae*, 2023).

This increasing availability of camera-trap images is facilitating the development of more accurate DL models to assist wildlife conservationists in several tasks (Christin et al., 2019). Norouzzadeh et al. (Norouzzadeh et al., 2018) applied multiple DL-based approaches on SS for (i) detecting animals, (ii) identifying species, (iii) counting the number of individuals, and (iv) describing animal actions. These authors trained various deep neural networks (DNNs) such as AlexNet, Network in Network, VGG, GoogLeNet, and ResNet, and explored different dataset sizes for network training and transfer learning. Gomez-Villa et al. (Gomez Villa et al., 2017) trained several DNNs for classification of wildlife species and explored how accuracy improves with the introduction of variations on the original data collection – balanced dataset, images containing only animals in foreground, images containing segmented individuals, etc. Beery et al. (Beery et al., 2018) addressed the problem of model generalization to new locations for animal classification. These authors concluded that training classification models on cropped regions of interest – i.e., bounding boxes – and using images taken at the same location during both training and test stages – so-called *cis-locations* –, are good practices in order to improve accuracy. Schneider et al. (Schneider et al., 2020a) also studied how the locations where the images composing a training dataset were taken affect network accuracy, and pointed out the importance of data augmentation and transfer learning techniques to improve training performance. Wang et al. (Wang et al., 2019) introduced anti-occlusion constraints during training to deal with occlusion in images taken in wild environments. Further studies on network training for particular species classification and binary empty–non-empty image filtering has also been reported (Gomez et al., 2016; Huang and Basanta, 2019; Miao et al., 2019; Nguyen et al., 2017; Parham et al., 2018; Tabak et al., 2019; Weinstein, 2018; Whytock et al., 2021; Willi et al., 2019).

As an advance with respect to image classification models, DL-based *object detection* networks (R-CNN, SSD, YOLO, etc.) can detect more than one individual per image and exhibit lower generalization error than classification models (Beery et al., 2018; Guo et al., 2019; Loos et al., 2018; Rigoudy et al., 2023; Schneider et al., 2018a; Singh et al., 2020). However, they demand more hardware resources and require manual annotation of the bounding boxes in the training images. More elaborated algorithms consist of multiple processing steps to build *animal detection pipelines* (Beery et al., 2019; Parham et al., 2018). The

exploitation of spatio-temporal data and background modeling from *video sequences* has also been proposed to detect and classify wild individuals (Chen et al., 2014; Chen et al., 2019; Miguel et al., 2016; Yousif et al., 2017; Yousif et al., 2019; Zhang et al., 2016). Provided that animal face datasets are collected, DNNs can be trained for *individual re-identification* as well (Schneider et al., 2018b). In particular, re-identification of primates and elephants has been addressed in this way (Brust et al., 2017; Deb et al., 2018; Freytag et al., 2016; Körschens et al., 2018; Schneider et al., 2020b).

All the related studies mentioned above were conceived for desktop analysis, as opposed to the focus of the present study, which is in situ analysis. Much fewer works can be found in the literature on this radical change in processing location that enables fast response times while reducing human labor and environmental impact. Classical computer vision algorithms, in conjunction with thermal and infrared sensors to filter out empty images, were tested on embedded devices (Tan et al., 2016; Ward et al., 2016). The capabilities of hardware platforms such as Raspberry Pi, Nvidia Jetson Nano, and Google Coral Dev for wildlife image classification have also been evaluated (S. C.K, S. E, 2019; Cunha et al., 2021; Curtin and Matthews, 2019; Dertien et al., 2023; Elias et al., 2017; Mathur and Khattar, 2019; Whytock et al., 2023; Zualkernan et al., 2022).¹ With the present study, we contribute to this specific research niche as follows: 1) we provide details about training of AI models adapted for integration in low-cost edge devices; 2) we propose two functional prototypes of low-power AI-enabled camera trap and report performance figures; 3) we give account of different field tests conceived to evaluate the system performance, and report on feedback model refinement based on the datasets resulting from these tests; the datasets are publicly available; 4) we define and thoroughly assess an advanced AI pipeline that incorporates on-device learning; 5) we detail learned lessons and identify challenges and opportunities for future work. In addition, for the sake of reproducibility (Huettmann and Arhonditsis, 2023), the datasets (Velasco-Montero and Sanglas, 2024) and code (Velasco-Montero, 2024) resulting from this study have been made publicly available.

2. Proposed methodology

As previously stated, our ultimate objective is to implement a smart camera trap that efficiently integrates AI to extract meaningful information from the inspected scene and allows quick actuation if necessary. To succeed in this endeavour, we propose the following methodology:

1. **Data collection and labeling:** Gathering images to construct a properly balanced dataset is mandatory when it comes to training a reliable DNN. These images must be manually labeled into categories, which is a time-consuming but essential task. In this study, we made the most of existing public datasets plus images registered by ourselves in the field using commercial camera traps.
2. **Network training:** Taking advantage of pre-defined network architectures already trained as a starting point avoids the need for training the network from scratch. Various actions can be carried out to improve model accuracy: dataset enhancement – e.g., applying data augmentation techniques –, introduction of suitable changes in the model architecture according to the application requirements, or fine adjustment of the training schedule.
3. **System integration:** Fitting complex AI pipelines on the limited resources of edge devices is a challenge. From the selection of hardware-efficient software libraries to the use, or not, of specific low-level coding techniques – e.g., multi-threading operation –, every decision along the way has a significant impact on the system performance, which makes this step particularly critical (Velasco-

¹ A comparison of state-of-the-art smart camera traps is provided in Section 4.5.



Fig. 1. Dataset I. Examples of the images captured by commercial camera traps at multiple locations of Sierra de Aracena Natural Park. The categories associated to these examples are *feral cat* (left), *other species* (middle), and *blank* (right).

Montero et al., 2022). This stage also includes the selection of any extra hardware components to be integrated with the main processing board.

4. **Comprehensive field testing:** Experimental validation of the system under realistic operation conditions is absolutely compulsory. It requires conducting a series of on-site tests in diverse real-world scenarios, which always pose new challenges and demand refinement until eventually meeting the prescribed specifications.

Steps (1) and (2) are covered in Section 3; Step (3) is addressed in Section 4; Step (4) is covered in Section 5; finally, as a result of the experience gained throughout the realization of these steps, Section 5 proposes an advanced AI pipeline, which represents the main contribution of this manuscript.

3. Preliminary analysis: Evaluation of off-site AI-based processing

3.1. Sierra de Aracena Natural Park (Dataset I)

The carnivore research group at Doñana Biological Station (EBD-CSIC) (Doñana Biological Station, 2024) studies population trends of various species in their natural habitats, with a focus on the population of feral cats in Sierra de Aracena Natural Park. Two members of this group, co-authors of the present study, have collected an extensive set of images and video sequences obtained from commercial camera traps (specifically, the Bushnell Trophy Cam HD Brown model 119874) strategically placed at different locations within the park. A significant portion of the collected data consists of ‘blank’ captures, containing no animal. Three distinct tasks, which directly translate into corresponding network classification objectives, are of interest for these biologists:

- Task 1 *Blank filtering*, to distinguish between blank images and those with relevant content. This is a binary classification task (2 classes: *animal* vs *blank*).
- Task 2 *Species identification*, to distinguish between feral cats and other species such as dogs, foxes, martens, and wild pigs. This is another binary classification task (2 classes: *feral cat* vs *other species*) that would constitute a subsequent stage after blank filtering.
- Task 3 *Concurrent species identification and blank filtering*, which implies direct classification of the images into three categories: *feral cats*, *other species*, and *blank*. Examples of these categories are illustrated in Fig. 1 (we name this collection Dataset I).

3.2. AI-based off-site processing

To gauge the potential of AI for the implementation of smart camera traps, we conducted preliminary tests using a desktop computer not to incur in the specificities of embedded hardware platforms yet. We selected DNNs, which constitute the state-of-the-art architecture for

Table 1

Preliminary study. Classification performance rates on Dataset I (several locations at Sierra de Aracena Natural Park).

Task	Train. / val. Accuracy	Val. Precision	Val. Recall	Val. F1-score
Task 1) Blank filtering	0.94 / 0.84	0.96	0.75	0.84
Task 2) Species identification	0.96 / 0.77	0.99	0.69	0.81
Task 3) Concurrent species id. & blank filtering	0.99 / 0.69	0.89	0.96	0.92

computer vision. In particular, we chose classification networks rather than object detectors owing to their lower computational load, which make them more suitable for edge devices. And more specifically, we selected the lightweight network SqueezeNet (Iandola et al., 2016). This model, according to our experience and previous work (Velasco-Montero et al., 2018), demonstrates superior energy efficiency and inference speed compared to other available pre-trained models when executing inference on our target embedded hardware (Section 4.1).

As a starting point, we used SqueezeNet pre-trained on the ImageNet dataset (Russakovsky et al., 2015). This instance of SqueezeNet was adapted to our problem by 1) replacing the network head (last 1000-category layer) with a classifier featuring the desired number of classes and 2) re-training.

We employed the popular TensorFlow-Keras package to re-train the network (Abadi et al., 2016). This software was run on Ubuntu 5.4.0 on a high-end computer featuring a GeForce RTX 2080 Ti 11GB GPU. For each of the three aforementioned vision tasks, Dataset I was split into training (80 %) and validation (20 %) datasets. During model training, input batches of images were augmented – applying zooming, rotation, width/height shifting, and horizontal flipping – using TensorFlow-Keras functions.

3.2.1. Training results

Dataset I. We used over 7000 images collected by different cameras on several locations of the park to create a well-balanced Dataset I. The neural network was trained by minimizing the cross-entropy loss between labels and predictions, utilizing the *RMSprop* algorithm for gradient descent (learning rate = 5×10^{-6}) on batches of 32 augmented images. After 25 epochs, we obtained the training-validation metrics reported in Table for the three tasks defined above.

In our calculations, we designate *animal* as the ‘positive’ class and *blank* as the ‘negative’ class to determine precision and recall rates.² It is important to emphasize that the optimization focus can be put either on precision or recall, depending on the specific needs of the end users – i. e., biologists, conservationists, natural environment managers, etc. A

² For Task 1, ‘positive’ class represents *feral cat* while ‘negative’ represents *other species*.

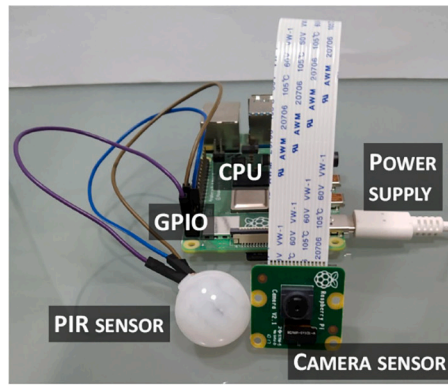


Fig. 2. Main hardware components of the first prototype of camera-trap vision system developed in this study (left) and picture of this prototype deployed in the field (right).

high precision rate indicates a low number of false positives (FP), which means reducing human labour; a high recall indicates few false negatives (FN), ensuring that we do not erroneously discard images containing individuals. In this study, both metrics were considered equally important; thus, F1-score – which combines precision and recall – is employed as a suitable indicator of performance. From the results in Table 1, we can conclude that SqueezeNet trained as described above achieves promising results in terms of inference performance.

In this study, we prioritized the embedded implementation of **blank filtering**, which, as we will see, is far from trivial. This task is crucial because it greatly reduces human labour in any wildlife scenario and is applicable to all species. In addition, to a lesser extent, the immediate removal of spurious images right after capturing precludes SD-card memory saturation, thereby extending the system lifetime, and alleviates the computational burden of subsequent off-site analysis. At this point, we must stress that although alternative approaches for blank filtering could be explored (i.e., image difference), our primary goal is the development of a *global AI-based solution*, with the intention of progressively incorporating additional network categories for species classification (e.g., Task 3 in Table 1).

Snapshot Serengeti Dataset. Collecting a large enough dataset for network training (such as Dataset I) is a resource-intensive and time-consuming task. In the absence of a custom dataset tailored for a particular wildlife scenario, publicly available *large datasets* could also be utilized. Their advantage lies in the fact that larger and more diverse datasets contribute to better model generalization. Therefore, in addition to the model trained as described in the previous section, we also trained SqueezeNet for blank filtering using SS data. Specifically, we utilized images from the first two seasons of SS to train the network (again with a distribution of 80 %–20 % training-validation). This involved creating a balanced large-scale dataset containing more than 400 k images. Following the fine-tuning of the entire SqueezeNet architecture (*RMSProp*, learning rate = 10^{-6} , batch = 128), we achieved accuracies of 0.85 and 0.80 in the training-validation datasets, respectively. We then evaluated the model trained on SS over the images of Dataset I, which were collected in a context significantly different than that of the African regions covered in SS. The accuracy of this model on Dataset I is 0.73, with a precision of 0.72 and a recall of 0.84 (i.e., an F1-score of 0.78), which indicates a good level of generalization.

3.3. Conclusions from preliminary analysis

- A lightweight network such as SqueezeNet is able to encode the knowledge required for accomplishing reliable wildlife visual inference.

- Starting from a pre-trained instance of this DNN on a dataset as diverse as ImageNet helps to reach convergence during re-training, even for a dataset as relatively small as Dataset I.
- The accuracy rates resulting from training SqueezeNet on SS and testing on Dataset I are promising in terms of model generalization. However, as we will see, additional actions are required to attain reliable field operation.

In this paper, we report results from field tests for SqueezeNet trained on both Dataset I and SS.

4. System design

4.1. Hardware

We built a first prototype of smart camera trap – Step (3) of Section 2 – based on Raspberry Pi 4B (Raspberry Pi 4, 2024), an embedded CPU-based processing platform. Raspberry Pi (RPI) denotes a series of small, inexpensive platforms that admit the connection of different components, including sensors and actuators. We integrated an off-the-shelf camera module (Raspberry Pi Documentation, 2024) into the system. For motion detection, we added a high-end passive infrared (PIR) sensor (Panasonic, 2024) that triggers image capture. For field deployment, the hardware set-up includes an external battery that supplies 5 V (model JIGA GN1 30,000 mAh). The resulting system with its different hardware components is presented in Fig. 2. Its bill of materials is approximately 100\$.

We employed this prototype for the tests reported in Section 5. In addition, we have built a second prototype based on a smaller, cheaper, and more energy-efficient embedded device: RPi Zero 2 W (Raspberry Pi Zero 2W, 2024). It features a compact form factor (65 mm × 30 mm × 5 mm, 11 g) and low power consumption (0.65 W when idle). Unlike the first prototype, in this second prototype we plan to integrate a real-time clock (RTC) module for accurate timestamping and a NoIR (no infrared filter) camera sensor for more efficient nocturnal operation. Although at the time of drafting this study we have conducted field tests only with the first prototype based on RPi 4B (Fig. 2), all our efforts are now put on the second one; we expect to report new experimental results in the near future.

4.2. Software

As part of Step (3) defined in Section 2, various software libraries, packages, and toolchains are required. Next, we provide a summary of the most relevant software components employed:

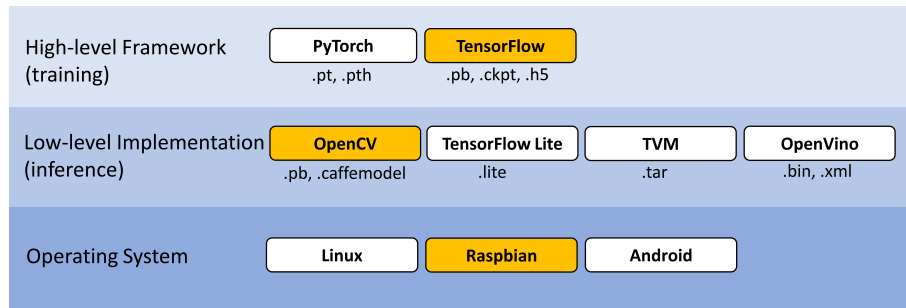


Fig. 3. Top-down workflow illustrating software components publicly available for training and integration of AI on embedded hardware. The components marked in orange were specifically selected for this work.

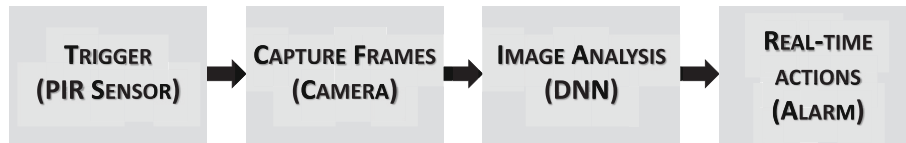


Fig. 4. Main operation stages of the prototyped smart camera trap: when the PIR detects movement, a set of frames is captured and analyzed by the DNN, thus enabling real-time actuation.

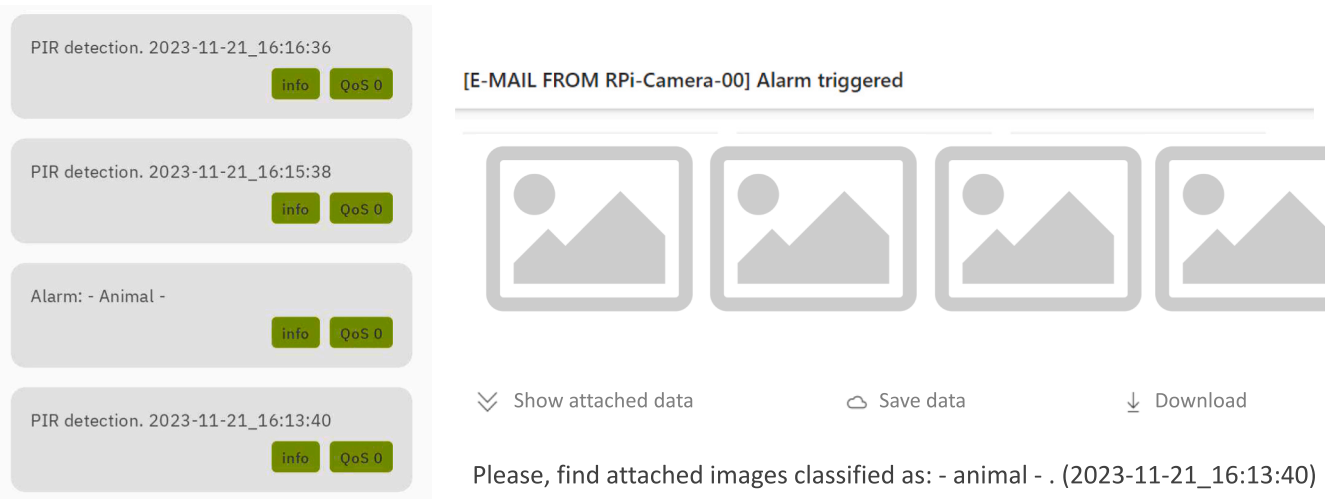


Fig. 5. Examples of messages sent by the camera trap and received in real time by a mobile phone using the MQTT protocol (left) and e-mail including wildlife images classified as *animal* (right).

- **Operating System.** Raspberry Pi OS (Raspberry Pi, 2024) (formerly known as Raspbian), specifically tailored for RPi series platforms, manages and integrates all software dependencies.
- **GPIO libraries** (Pigpio Library, 2024). They are used to access the general-purpose input/output (GPIO) pins for interaction with the PIR sensor; these pins can also be used to control other external sensors and/or actuators.
- **OpenCV** (OpenCV, 2024). Popular library used for accessing the camera and pre-processing captured images.
- **Inference libraries.** The network weights trained with TensorFlow (TF) are typically stored in formats such as checkpoint (.ckpt), SavedModel (.pb), or HDF5 (.h5). Then, these weights need to be transformed to another format for embedded inference. The regular procedure is to transform the model to TensorFlow Lite format (.tflite). Additionally, we also explored OpenCV for network inference.

To boost performance through parallelization on the Quad Core ARM processor of the RPi, the compilation of both OpenCV and TF Lite was configured to exploit the ARM NEON architecture – providing SIMD (Single Instruction Multiple Data) instructions – and ARM VFP (Vector Floating-Point) hardware registers.

- **Communication library.** The system was endowed with alarm/alert functionality. For now, we make use of the Wi-Fi protocol, although other communication protocols can be easily integrated. Specifically, the camera can send messages using the Message Queuing Telemetry Transport (MQTT) protocol or an e-mail service. For MQTT, we installed the MQTT broker. Thus, the embedded application acts as an MQTT client to send information messages (publisher). A remote device, e.g., a mobile phone, could act as a client (subscriber) to receive messages from the camera.

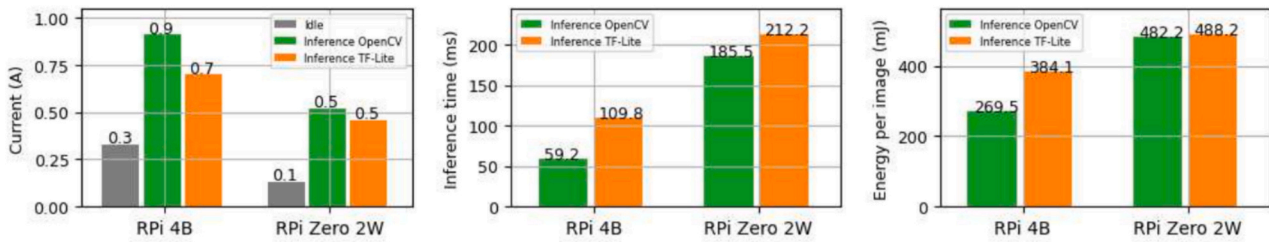


Fig. 6. Inference performance of 2-category SqueezeNet running on different software libraries and two embedded platforms: RPi 4B and RPi Zero 2 W. **Left:** Current consumed when performing inference; **middle:** Inference time including image loading and pre-processing; **right:** Corresponding average energy per image when performing inference.

Fig. 3 summarizes the essential libraries at different levels required for training and integration of AI on an edge device (not RPi necessarily).

4.3. Embedded application

Both prototypes have been designed to provide the basic functionality of a commercial camera trap, coupled with on-device image processing for scene analysis and decision making. The workflow of the first version of embedded application is illustrated in Fig. 4. It was encoded in C++ to enhance execution performance in conjunction with the aforementioned acceleration back-end libraries.

The system awaits for PIR activation due to motion in the surroundings. This motion can be caused by the presence of an animal – ideally – or by spurious motion – typically tree leaves or abrupt illumination changes in the scene. Upon PIR activation, the following stages are immediately executed:

- **Frame capture.** The camera captures multiple frames (“burst”) after each trigger event. Capture parameters can be configured by the user, as explained below.
- **DNN inference:** AI is implemented through a DNN previously trained, thus giving rise to the concept of *smart* camera trap or, equivalently, AI-enabled camera trap.
- **Real-time actions.** As mentioned earlier, the prototype supports two types of actions: e-mail notification and MQTT messaging. In the case of using the e-mail service, an e-mail is sent to a prescribed destination if images are classified as relevant (*animal*), attaching the images, camera location, and timestamp.³ If MQTT messaging is activated, notifications on various aspects of the system can be provided, e.g., discarded PIR activations. Fig. 5 provides examples of messages sent by the camera, offering real-time insights about its environment. It includes PIR events filtered out as *blank* by the DNN and one PIR event classified as *animal*.

The code for this embedded application, along with some comments to facilitate its use, is available at (Velasco-Montero, 2024).

Configuration parameters. The operation of the smart camera can be adjusted to the user preferences through configuration parameters. A configuration file allows defining various settings such as image resolution, burst duration, frame rate, relevant class (e.g., *animal*), DNN file, class-label file, and camera name, among others. By default, the resolution of the captured images is set to 320×240 , which is a standard resolution – compatible with camera-access libraries of OpenCV – close to that of the DNN input (227×227). After a motion event triggered by the PIR, the camera takes a burst of 10 frames at a frame rate of 2 frames per second (fps), high enough to register quick animal passing in most

practical scenarios. In any case, note that these parameters are fully configurable.

4.4. System performance

Fig. 6 shows lab performance measurements of both aforementioned prototypes – based on RPi 4B and RPi Zero 2 W – and two inference libraries – OpenCV and TensorFlow Lite – when executing SqueezeNet. The current consumed (Fig. 6–left) represents the maximum value observed from measurements with the Keysight E36311A device applying a 5-V input voltage. The inference time (Fig. 6–middle) includes image loading and pre-processing in addition to the DNN execution itself.⁴ The average energy consumption per image (Fig. 6–right) is derived from the other two metrics. In each plot, lower values indicates better performance. More details about system performance are summarized next:

- **Power consumption (Fig. 6–left).** For a 5-V power supply, the RPi-Zero-based system consumes approximately 0.13 A (0.65 W) in idle state and 0.45–0.53 A (2.25–2.65 W) during DNN image processing, whereas the RPi-4B prototype consumes 0.33 A (1.65 W) in idle state and up to 0.91 A (4.55 W) during inference. Some additional comments:
 - The camera will spend most of the time in the idle state awaiting for an event. Therefore, low power consumption during this state is crucial (Fig. 6–left, grey bars). This definitely favors the selection of RPi Zero as the base hardware platform.
 - OpenCV, characterized as the optimal choice for RPi 3B in a prior study of ours (Velasco-Montero et al., 2018), presents similar performance to TensorFlow Lite in terms of energy consumption per image in the RPi Zero for this 2-category SqueezeNet model (Fig. 6–right, rightmost green and orange bars). For RPi 4B, OpenCV consumes 30 % less energy per image than TensorFlow Lite (Fig. 6–right, leftmost green and orange bars).
 - The current consumption of the PIR sensor is 6 μ A, almost negligible with respect to the total current consumption of the system. We also observed during field testing that this sensor reduces the number of captured and processed images by 81 % with respect to a possible alternative configuration without PIR sensor and image analysis at a constant rate of 2 fps (see Section 4.3) – this calculation contemplates the 35 k images captured in the field versus the number of images that would have been captured at 2 fps throughout the entire field test. In other words, the integration of a PIR sensor is critical for energy efficiency, as expected.

³ In the absence of an RTC, we manually set the initial time of the system using the configuration file.

⁴ As usual for DNNs, the input data must undergo pre-processing consistent with the training set-up. For our network, this includes mean subtraction, scaling, and data rearrangement into the desired BCHW (batch size, number of channels, height, width) 4-D array format.

Table 2
State-of-the-art AI-based smart camera traps.

Work	Hardware	AI model & software	Main features
Whytock et al. (Whytock et al., 2023)	Bushnell camera trap + RPi 4 compute module	3-category model (not described) (elephant-human-other), TF Lite	✓ Real-time alerts (inference: ~ 100 ms) ✗ High cost
Dertien et al. (Dertien et al., 2023)	TrailGuard AI prototype (Intel Myriad 2 computer vision chip)	Customizable AI detectors	✓ Real-time alerts (inference: ms) ✓ Low power, small size
Elias et al. (WTB) (Elias et al., 2017)	Distributed IoT architecture (cloud tier + edge units + Reconyx cams)	4-category (bear-deer-coyote-empty) Inception-v3, TF/OpenCV	Cloud training, edge inference ✗ High cost
Zualkernan et al. (Zualkernan et al., 2022)	Bushnell cam trap + edge node (RPi 4, Google Coral, Nvidia Jetson Nano)	Various classifiers (15 categories), TF/TensorRT	✓ Fast response (276 ms), ✗ high cost ✓ Low power (1.66 A), small size
Cunha et al. (Cunha et al., 2021)	RPi 3B	Various detectors and classifiers, TF Lite	✗ Complete camera-trap system not built ✓ Real-time alerts (~ms), low cost
This work	RPi 4B RPi Zero 2 W	2-category SqueezeNet (animal-blank), TF/OpenCV	✓ Ultra low-power, small size

- System autonomy will largely depend on the specific deployment conditions and requirements. For instance, both prototypes can be powered by a solar panel to further extend the system operation time, but note that solar panels may be impractical in areas prone to vandalism or with limited access to sunlight owing to dense vegetation. Additionally, if the target species is active only during certain times, e.g., dawn or dusk, the prototypes can be configured to operate exclusively during those periods, which would significantly extend the battery lifetime.

- **Processing time.** The average inference time per image (including image loading, pre-processing, and DNN inference) is approximately 185 ms on the RPi Zero. This translates to less than 2 s of image processing per captured burst (10 images/burst), thereby enabling provision of real-time actuation.
- **Memory:** the DNN accounts for 2.78 MB on the SD card, which can be considered negligible for the current capacity of such cards. Therefore, the capacity available for collecting wildlife images is hardly affected.

4.5. Comparison with state-of-the-art systems

Different flavors of RPi-family devices have been increasingly employed for ecological research in a diversity of application contexts, including underwater video surveillance, electronic animal sensing, and plant phenotyping (Jolles, 2021).

Notwithstanding, as mentioned in Section 1.1, few studies have focused on the realization of AI-enabled camera traps. Table 2 provides a comparative analysis of the reported systems in technological terms.⁵ Among these, Whytock et al. (Whytock et al., 2023) proposed the most similar solution to ours: a commercial camera trap (costly compared with our solution) modified to incorporate AI capabilities through a coupled RPi-based device and a satellite modem. No software

optimization was implemented. Similarly, Zualkernan et al. (Zualkernan et al., 2022) proposed an Internet-of-Things (IoT) architecture including a Bushnell camera trap, an edge device for species classification, and a LoRa communication module. Various edge platforms and DNNs were compared. Another interesting prototype is TrailGuard (Dertien et al., 2023), which relies on several object detectors tailored for specific species – details about the inference models have not been reported. The AI model stored in the SD card integrated in TrailGuard must be manually changed according to particular application requirements. “Where’s the bear?” (WTB) (Elias et al., 2017) is a multi-tier system designed for species classification. The DNNs implemented in this device are adapted to the specific locations of camera deployment. To this end, network training is executed on the cloud – using images from the deployed cameras –, and the resulting model is transferred to the edge. This system incurs high costs derived from the complex IoT architecture required. Another related work (Cunha et al., 2021) proposed both edge hardware and DNN models for wildlife image classification, but a complete camera-trap system or prototype was not developed.

In this comparison, our smart camera traps stand out for their compactness and inexpensiveness. In addition, by integrating libraries optimized for the underlying hardware, we accomplish energy efficiency. Our ultimate objective is model generalization, that is, we aim to create a system capable of accurately detecting individuals at any wildlife location – this is also the ultimate goal of WTB (Elias et al., 2017). Steps in this direction are described in Section 5.

5. On-site AI-based inference

5.1. Dataset II

It is usually assumed that models trained off-site achieve a performance in real-world scenarios in line with the accuracy of the validation dataset. However, creating a good offline model is just the initial phase, as will be shown next.

Field testing. After building the first prototype of smart camera trap based on RPi 4B and embedding SqueezeNet trained on Dataset I, we conducted field tests at a specific location in the Sierra de Aracena Natural Park. This location was selected because of the high probability of feral cat passing. Interestingly, this location had not been covered by any of the commercial camera traps that provided images for Dataset I. Fig. 7 shows examples of images at this location, previously unseen by the trained DNN. This allowed us to evaluate the system adaptability to untrained scenarios, providing valuable insights for further refinement and optimization. No animals other than feral cats were observed at this location.

The camera was configured with the standard configuration previously described: A burst of 10 frames were analyzed on-site by the DNN whenever motion was detected by the PIR sensor. Over a period of approximately 26 h of continuous operation in the field, the camera captured over 35 k images (more details on this later on). Out of these images, we manually classified more than 10 k images, giving rise to ‘Dataset II’. This dataset, though unbalanced (13 % cats – 87 % blanks), covers a wide range of illumination and vegetation variations, as well as many distinct situations – e.g., individuals located far away from the camera or partially occluded.

5.1.1. First approximation: Off-site trained model, real-world on-site testing

Model trained on Dataset I. The DNN trained on Dataset I (see Section 3.2.1) applied to these manually labeled images (Dataset II) produced the results presented in the first row of Table 3.⁶ Its confusion matrix is provided in Table 4. Note that the fact that Dataset II is unbalanced

⁶ Note that Dataset I — as well as SS — comprises both day and night images, unlike Dataset II. Therefore, the model trained on Dataset I features greater generality and is applicable to Dataset II.

⁵ In terms of accuracy, the comparison is provided in Section 5.4.



Fig. 7. Dataset II. Images captured by our first prototype of smart camera trap during field testing at a specific location – previously unseen by the trained DNN – in the Sierra de Aracena Natural Park. The corresponding categories are *animal* (left) and *blank* (right).

Table 3

Classification performance rates of SqueezeNet trained and tested on different datasets.

Training dataset → Test dataset	Precision	Recall	Accuracy	F1-score
Dataset I → Dataset II	0.19	0.81	0.52	0.31
SS → Dataset I	0.72	0.84	0.73	0.78
SS → Dataset II	0.13	0.92	0.18	0.23

Table 4

Results of model trained on Dataset I when tested over on-site captured images (Dataset II).

	Predicted Animal	Predicted Blank	
Presence	1071	250	Recall = 0.81
Absence	4581	4218	
	Precision = 0.19		Acc. = 0.52, F1-score = 0.31

makes accuracy less informative than precision and recall (and consequently, F1-score). These results, characterized by a low precision due to many FP, highlight the performance loss associated to performing inference at an unseen location under changing environmental conditions (Beery et al., 2018).

Explainable DL. To delve deeper into the DNN operation and its performance, we leveraged *explainable DL* techniques (Xie et al., 2022); specifically, we applied the Grad-CAM algorithm⁷ (Selvaraju et al., 2017). Both Figs. 8 and 9 show two images with their corresponding explanation map on their right side. The following observations can be made:

- Most FP in this scenario were due to network misinterpretation of vegetation changes as cat individuals. See, for instance, a fallen leaf in the foreground in Fig. 9–left (initially not there; see images in Fig. 7). This leaf, present in a high number of theoretically *blank* images, was incorrectly identified by the DNN as an animal.
- Regarding FN, the difficulty of on-site inference at this location is evidenced by the presence of distant or semi-occluded individuals, which is challenging even for the human eye. (Fig. 9–right).

⁷ Grad-CAM (Selvaraju et al., 2017) backpropagates the gradients of the target class in the last convolutional layer to capture the importance of feature maps for the target class. By combining the importance of feature maps, it produces a coarse localization heatmap containing the relevant regions in the input image for predicting that class. This algorithm has already been employed in similar ecological studies (Miao et al., 2019).

Model trained on SS. When applying our SS-trained model (see Section 3.2.1) to Dataset II, we obtain the results shown in the third row of Table 3. While the results for this same model applied on Dataset I (second row in that Table) may be acceptable, as discussed earlier, when applied to Dataset II, we observe a noticeable loss in accuracy and precision rates. This is highlighted by the confusion matrix, reported in Table 5. The reason for this high number of FP is that the DNN must deal with an unseen and challenging location in Dataset II. This location is characterized by a confusing vegetation background and extreme illumination variations (Beery et al., 2018). This clearly calls for additional steps to address the challenges posed by environmental variations, such as endowing the model with further information about the particular scene under inspection.

5.2. Proposed approach

Based on the insights gained during the field tests reported in the previous section, in particular those concerning the deficiencies of the direct application of a trained model forced to operate at unseen locations, we next propose a methodology that accomplishes better results. It is based on two key refinement points: (1) on-device training exploiting location-specific data, and (2) continual learning (De Lange et al., 2022; Lesort et al., 2020).

5.2.1. On-device learning

We aim to incorporate knowledge about the specific camera location into the DNN model. To this end, after exploiting a large-size dataset such as SS for off-site training, we apply *transfer learning* (TL) (Zhuang et al., 2021) – which is specially advantageous when the targeted inference scenario shares similarities with the original one – through *on-device training*, whose realization at the edge is remarkably challenging but definitely beneficial.

5.2.1.1. Location-specific dataset. Given that there is no dataset available for unseen locations, we produce a synthetic one on the fly. The images composing this dataset are generated on-site by the device to incorporate knowledge about the background captured by the camera at a specific location. Background images collected on-site when the PIR does not detect movement make up the ‘Blank’ category. Likewise, cropped individuals of the species of interest, i.e., feral cats in this case, are synthetically placed on this background, to create the ‘Animal’ class. The generation of this dataset includes variations in position and size of

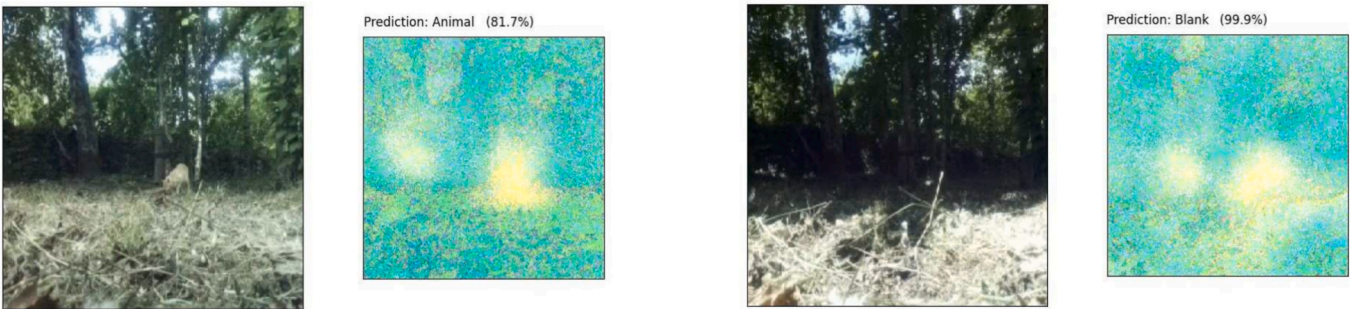


Fig. 8. Grad-CAM (Selvaraju et al., 2017) explanation maps for a TP sample (left) and a TN (right) image. Yellowish pixels correspond to the areas that have a greater influence on the DNN output (text over the image).



Fig. 9. Grad-CAM explanation maps for a FP example (left), and a FN sample (right) – there is a cat in the area marked in red, though hardly distinguishable. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

Table 5
Results of the model trained on SS when tested over on-site captured images (Dataset II).

	Predicted Animal	Predicted Blank	
Presence	1221	100	Recall = 0.92
Absence	8221	578	
	Precision = 0.13		Acc. = 0.18, F1-score = 0.23

the individuals, as well as other parameters related to data augmentation. Examples of synthetically-generated training images of the ‘Animal’ category are shown in Fig. 10.

5.2.2. On-device training
The updated workflow for the application embedded on the smart camera trap is illustrated in Fig. 11. It includes an initial ‘calibration period’ in which the camera acquires knowledge about its environment once deployed at a specific location. During this calibration, (i) the camera takes a number of background images (*blank*),⁸ (ii) generates synthetic images containing individuals (*animal*), and (iii) re-trains the network (TL from SS) with these two categories of data. Subsequently, the resulting trained network serves as the core AI in the main workflow (‘camera operation’ in Fig. 11). As previously mentioned, the integration of this training process into embedded hardware is extraordinarily challenging owing to the limited resources available. Nevertheless, we integrated the TensorFlow library for on-device training into both pro-



Fig. 10. Synthetic training data for the *animal* class generated on-site by combining the background of the location with previously cropped individuals.

⁸ Note that this means that the user of the system must ensure that no animal passes in front of the camera while this initial calibration takes place.

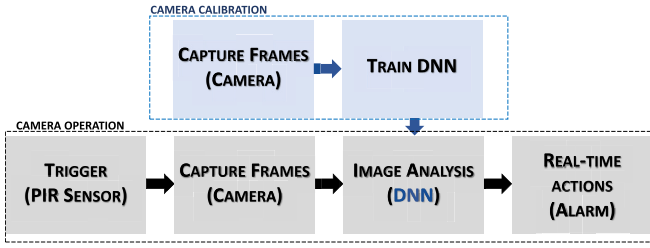


Fig. 11. Enhanced smart camera operation. The network is trained on-site with a dataset of images captured at the deployment location.

totypes and satisfactorily checked its execution feasibility.

To assess the effectiveness of the proposed method before integrating it in the camera and conducting field testing, we performed simulations on a PC. To this end, we made the most of the fact that, during previous field tests, we periodically – every second – stored images even if no event had been detected by the PIR. This vast image collection was not part of the actual camera operation; it was intended just for research, testing, and experimentation; and indeed, it proved to be very useful, as will be shown next. We employed a set of 240 background images collected during the initial minutes of camera operation to generate a balanced synthetic training dataset (480 images in total, half of them including synthetically added individuals). We fine-tuned the DNN already trained on SS, employing batches of 16 images with the *RMSProp* optimizer (learning rate = 10^{-5}), achieving a training accuracy of 0.80.

For this first approximation of initial calibration phase, testing over Dataset II resulted in values of precision and recall of 0.13 and 0.99, respectively, with an accuracy of 0.14 and F1-score of 0.23. This means that the model still misinterprets some scene contents as individuals (FP). The reason lies in the changes in illumination, vegetation, etc. at this location, which were out of the scope of the changes reflected in the training dataset during the first minutes of system operation – see for example the fallen leaf in Fig. 9, which was not initially there at the time of the first calibration. This means that the camera still faces what is known as *data drift*, that is, changes experienced by the input data distribution over time that degrades the network accuracy (Ackerman et al., 2020). We address this issue in the next subsection.

5.2.3. Continual learning

Continual learning (CL) is a paradigm that assumes that the data distribution or learning objective changes over time. CL systems learn from a stream of non-stationary data (so-called ‘experiences’ (Lesort et al., 2020)), also with the aim of optimizing the training process without requiring massive computation for each new experience (Cossu et al., 2021; De Lange et al., 2022; Lesort et al., 2020). In our case, we face a so-called single incremental task with new instances, meaning that the data observations change over time but relate to the same task (blank filtering) (Lesort et al., 2020).

Among state-of-the-art CL strategies, we propose a *rehearsal* approach, which is a general strategy for CL realization. This strategy is based on retaining knowledge about past experiences (raw samples), specifically a limited number of samples – for instance, randomly chosen –, which should be representative of the past. Therefore, we propose periodically re-train the DNN using batches of synthetic data generated from temporally distributed captured backgrounds (*basic random*

replay). We can reduce the training computation by freezing certain layers and employing a reduced training dataset (Pellegrini et al., 2020).

Specifically, utilizing the experimentation data gathered in the field, we simulated this replay method by incorporating a second experience as follows:

1. First calibration (‘train-01’), already described in Section 5.2.1. In this case, the training data comprise images collected during the initial minutes of camera operation. TL leverages the model trained on SS.
2. Second calibration (‘train-02’), conducted on data registered several hours later. The data for this second experience include images collected in the time interval between ‘train-01’ and ‘train-02’. TL is now applied on the model trained in previous experience (‘train-01’). To prevent *catastrophic forgetting* (French, 1999) in this second calibration, we implemented *cumulative random replay* (Maltoni and Lomonaco, 2019), employing 30 % images of past experience (experience ‘01’), and 70 % from the new experience (experience ‘02’), amounting to $N = 480$ images (randomly selected) for this second training. For re-training, we unfroze only the last 5 *Fire* modules of SqueezeNet and conducted training over 5 epochs. The training accuracy achieved in this case was 0.84.

The testing datasets corresponding to these two experiences were obtained by partitioning the original Dataset II as follows:

Table 7

Summary of DNN hyper-parameter tuning for in-situ training.

	5 epochs				Comment
	Precision	Recall	Accuracy	F1-score	
Batch = 16 •	0.69	0.36	0.90	0.48	Highest F1-score
Batch = 32	0.59	0.36	0.88	0.45	Close performance
Batch = 64	0.36	0.41	0.83	0.38	Performance drop
RMSprop •	0.69	0.36	0.90	0.48	Highest F1-score
Adam	0.77	0.32	0.89	0.45	Close performance
SGD (lr= 10^{-3})	0.54	0.42	0.88	0.47	Close performance
Nadam	0.67	0.35	0.89	0.46	Close performance
Learning rate = 10^{-6}	0.14	0.55	0.51	0.23	Does not converge
Learning rate = 10^{-5} •	0.69	0.36	0.90	0.48	Highest F1-score
Learning rate = 10^{-4}	0.41	0.38	0.84	0.40	Performance drop
80 imgs/categ	0.13	0.97	0.13	0.23	Does not converge
160 imgs/categ	0.43	0.43	0.85	0.43	Performance drop
240 imgs/categ •	0.69	0.36	0.90	0.48	Highest F1-score
320 imgs/categ	0.74	0.35	0.90	0.48	Close performance

Table 6

Train-test accuracy matrix, including additional classification metrics, resulting from implementing CL over two experiences.

	test-01				test-02				Dataset II			
	Precision	Recall	Accuracy	F1-score	Precision	Recall	Accuracy	F1-score	Precision	Recall	Accuracy	F1-score
train-01	0.11	0.99	0.11	0.19	0.33	0.99	0.33	0.50	0.13	0.99	0.14	0.23
train-02	0.58	0.21	0.90	0.31	0.81	0.80	0.87	0.81	0.69	0.36	0.90	0.48

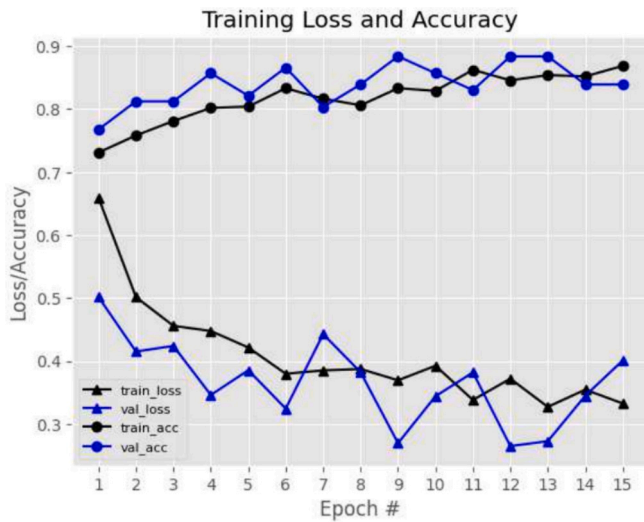


Fig. 12. Evolution of cross-entropy loss and accuracy during network training.

1. 'test-01'. Set of labeled images captured on-site before the 'train-02' time.
2. 'test-02': Remaining labeled samples collected on-site after the 'train-02' time.

The resulting train-test matrix R_{ij} (Rodríguez et al., 2018) reporting the accuracy of each network instance over different data experiences is shown in Table 6. Note that this matrix includes not only accuracy but also other classification rates. The results for 'train-01→Dataset II' were discussed in Section 5.2.1. The results of 'train-01→test-02' are related to the forward transfer metric (testing on future experiences), whereas the values of 'train-02→test-01' correspond to the backward transfer metric (testing on past experiences) (Rodríguez et al., 2018). Table 6 confirms that CL enables a better understanding of the wildlife scene inspected by the DNN as well as adaptation to the unavoidable changes occurring in natural environments over time.

5.3. DNN training: Hyper-parameter and dataset tuning

We have just shown that the incorporation of in-situ training and continual learning boosts inference metrics. For the practical implementation of these techniques on the smart camera trap, we explored different training settings to determine the optimal configuration. Initially, we considered a batch size of 32, using RMSprop with a learning rate of 10^{-5} and 240 training images per category. Sequentially, we altered one parameter at a time and assessed its impact on inference metrics. The outcomes are summarized in Table 7.⁹ The optimal selection in terms of F1-score is marked with dots in the table. These results correspond to the 'train-02→Dataset II' training process. Further details are provided next:

- **Number of epochs:** A typical example of training process evolution is depicted in Fig. 12. Concerning on-device training, there is an important trade-off between the number of training epochs and energy saving. Therefore, although training-validation accuracy might continue increasing with additional epochs, we opted to limit our study to 5 epochs. Note that the initial epochs result in significant accuracy improvement compared to latter epochs, which mainly refine the training process.

⁹ Note that the stochastic nature of the training process introduces slight variability in the results across different training executions.

Table 8

State-of-the-art algorithms used for a comparative study (presented in Table 9).

Approach	Model	Training Dataset	Reported Acc.
MegaDetector (MDv4) (Beery et al., 2019; GitHub Microsoft, 2024)	Faster-RCNN	multiple datasets	not reported ^a
DeepFaune (DeepFaune, 2024; Rigoudy et al., 2023)	YOLOv8s	own dataset	0.79, 0.68 ^b
Willi et al. (Willi et al., 2019; Willi et al., 2024)	ResNet-18	Camera CAtalogue	0.98
Norouzzadeh et al. (Task I) (Norouzzadeh et al., 2018)	VGG	SS	0.97

^a Authors do not report accuracy in the manuscript; however, they state that the "performance of any AI system can vary in new environments" (GitHub Microsoft, 2024).

^b Reported detection of 79.6 % over 119,831 images containing large animals, 66.2 % over 7036 images containing small animals, and 2.97 % over 8583 negative samples. This corresponds to an accuracy equal to 0.79 for large animals and 0.68 for small individuals.

Table 9

Performance of state-of-the-art algorithms over Dataset II.

Approach	Precision	Recall	Accuracy	F1-score
MegaDetector (Beery et al., 2019; GitHub Microsoft, 2024)	0.28	0.88	0.70	0.43
DeepFaune (DeepFaune, 2024; Rigoudy et al., 2023)	0.96	0.35	0.91	0.51
Willi et al. (Willi et al., 2019; Willi et al., 2024)	0.31	0.43	0.80	0.36
Norouzzadeh et al. (Task I) (Norouzzadeh et al., 2018)	0.13	0.99	0.16	0.24
This work (CL, train-02→Dataset II)	0.69	0.36	0.90	0.48

- **Batch size:** we considered three different batch sizes (16, 32, and 64). As shown in Table 7, a batch size of 16 images is clearly the best option and was the value set for the next iteration of our study. Note that larger batch sizes consume more RAM memory, which could potentially hinder network training on the RPi Zero or even necessitate the use of swap memory, resulting in inefficiency.
- **Optimizer:** we compared four optimization algorithms: RMSprop, Adam, SGD,¹⁰ and Nadam. The results presented in Table 7 indicate that our initial choice, RMSprop, ensures appropriate convergence.
- **Learning rate:** inference metrics exhibit a remarkable sensitivity to this parameter. A value of 10^{-5} clearly leads to the best performance in terms of F1-score.
- **Number of training images:** we built different training datasets of varying sizes. Larger datasets require more training resources. To attain acceptable results, we selected 240 images per category.

As a final comment, the training time for the selected hyper-parameters on RPi 4B is approximately 8 min whereas on RPi Zero 2 W it is approximately 15 min (these intervals vary depending on the ambient temperature at the deployment location (Benoit-Cattin et al., 2020)) – for comparison, the same training on the high-end computer described in Section 3.2 takes 30 s. These short training times in platforms featuring so limited hardware resources are achieved through a reduced number of trainable weights (around 700 k) and epochs, and a small training dataset. We also measured the power consumption of on-device training, which resulted in a maximum current demand of approximately 0.7 A in both prototypes. This corresponds to approximately 93 mAh for the RPi 4B and 175 mAh for the RPi Zero 2 W, accounting for less than 0.6 % of our battery capacity (see Section 4.1).

¹⁰ Each optimizer converges at a different pace. Therefore, to ensure convergence of SGD, we employed a higher learning rate.



Fig. 13. Left: sample image analyzed with MegaDetector (GitHub Microsoft, 2024). Middle-right: true positive (middle) and false negative (right) samples analyzed with DeepFaune (Rigoudy et al., 2023) (these two examples correspond to the same individual).

The code available at (Velasco-Montero, 2024) also includes a fully configurable implementation of this continual learning approach.

5.4. Comparison with state-of-the-art algorithms

Locally captured images, different in essence to that of public datasets, suffer from data drift (environmental changes) that cause many AI classification errors. This is alleviated by the proposed approach based on on-device continual learning (Sections 5.2.1 and 5.2.2), which achieves better results than off-site training without re-training (Section 5.1.1).

As an example of the difficulties of processing field wildlife images, we conducted tests using state-of-the-art algorithms for comparison. In particular, we selected MegaDetector, which is a well-known detection model, as well as other algorithms reported in related works that publicly provide their trained models, as detailed in Table 8. The second column of this table shows that these approaches employ DNNs that demand significant computational resources, resulting in low frame rates even in GPU-based systems, which virtually discards their execution on embedded systems.

The accuracy rates obtained when applying the models in Table 8 to Dataset II are summarized in Table 9.¹¹ Examples of inference with these publicly available models are shown in Fig. 13. This figure depicts output bounding boxes corresponding to detector models in Table 8, and its only purpose is to illustrate examples of unexpected behavior from these techniques. Note that MegaDetector struggles to correctly identify the background, misinterpreting fallen leaves as individuals (Fig. 13, left image), while DeepFaune also misses distant individuals (Fig. 13, middle and right images). Despite being large-size detection models trained on millions of images, they incur in unexpected classification errors.

An analysis of the numerical data in Table 9 reveals that MegaDetector and Norouzzadeh's models tend to classify images as *animal* (owing to background misinterpretation), resulting in high recall but at the expense of low precision. In contrast, DeepFaune correctly identifies animals in the scene, accomplishing high precision and modest recall. Our approach – suitable for embedded systems, unlike the other algorithms – achieves a balanced performance in both metrics, with reasonable precision and moderate recall, resulting in a comparatively high F1-score.

6. Learned lessons: Challenges and opportunities

Next, we briefly summarize some challenges and research opportunities identified during the realization of the present study.

- Accurate off-site training does not always translate to good performance in real field operation. Poor network generalization leads to

many classification errors. To address this issue, we propose a solution based on acquiring location-specific information (Section 5.2).

- Explainable DL is an effective method to evaluate network operation and identify potential deviations in the attention of the DNN that mislead inference.
- Real-world scenarios pose challenges due to abrupt illumination changes, motion of vegetation, etc. Even methods based on massive training fail when tested in such scenarios (Table 9).
- The same individual in slightly different positions can lead to variations in the network prediction (Fig. 13, middle and right images). A potential solution could be applying prediction on a sequence of images rather than individual images.
- We observed that the network confidence for a substantial number of inference outcomes is low (around 50–60 %). This suggests that a confidence threshold could be applied to improve the results (Gomez et al., 2016; Willi et al., 2019).

This manuscript is the first milestone in our long-term research aiming at the realization of smart camera traps capable of automatically adapting to the environment and generating meaningful periodic reports over long periods of autonomous operation. Concerning **data privacy and ethical operation**, one advantage of incorporating intelligence directly into the device itself is the ability to implement measures such as person filtering to preserve privacy or person detection to identify the presence of poachers. In general, the capabilities of the proposed system can be put, as far as possible, at disposal of the realization of codes of conduct such as the one outlined in (Sharma et al., 2020) for use of camera traps in wildlife research.

As future research lines, we will conduct additional field tests to collect more data that will enable optimal selection of hyper-parameters using an off-site cross-validation scheme. Additionally, we aim to explore various CL strategies, taking into consideration accuracy metrics and hardware resource consumption (e.g., replay buffer, training time, memory, etc.). The use of temporal information (that is, video analytics) and training of different network architectures is also planned. Adjusting parameters to prioritize precision or recall, based on particular application requirements, will be considered. Multi-class species classification will also be studied to be integrated on the system. Once the lines outlined above are explored and the system becomes robust enough, collaborative networking can boost the application scope of smart camera traps.

7. Conclusions

AI is generating high expectations in each and every technical field, and conservation technologies are not an exception. While the advances in automatic analysis underpinned by AI cannot be denied, there is still a long way to go. In this paper, we show that incorporating a task as apparently simple as blank filtering to a camera trap is far from trivial. The challenges are not only coming from the meager hardware resources available in such a system, but also from the completely uncontrolled scenarios in which it must operate. We demonstrate that even DNNs specifically and massively trained for species identification are not

¹¹ Although some works also provide multi-class classification or animal detection, we focus here exclusively on analyzing blank-vs-animal classification rates.

reliable when applied to some of the images we collected in a real-world location. This location was not expressly selected for its difficulty, which anticipates similar issues in regular deployments of camera traps. We consider that the proposed strategy, based on leveraging public datasets, TL applied to locally generated data, on-device training, and CL, establishes a solid basis to accelerate the incorporation of intelligence to camera traps.

CRedit authorship contribution statement

Delia Velasco-Montero: Writing – review & editing, Writing – original draft, Validation, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Jorge Fernández-Berni:** Writing – review & editing, Writing – original draft, Methodology, Investigation, Funding acquisition, Formal analysis, Conceptualization. **Ricardo Carmona-Galán:** Writing – review & editing, Investigation, Funding acquisition. **Ariadna Sanglas:** Writing – review & editing, Investigation, Data curation, Conceptualization. **Francisco Palomares:** Writing – review & editing, Investigation, Funding acquisition.

Data availability

The links to the datasets and code developed in this study are included in the paper. Both the datasets and code are publicly available.

Acknowledgments

This work was supported by project SEMIoTICS (PID2021-128009OB-C31), funded by MCIN/AEI/10.13039/501100011033 and EU “ERDF: A way of making Europe”; by project ULTIMATE (TED2021-131835B-I00), funded by MCIN/AEI/10.13039/501100011033 and by the European Union NextGenerationEU/PRTR; by project SUMHAL (LIFEWATCH-2019-09-CSIC-4, POPE 2014-2020), funded by Ministry of Science and Innovation of Spain through European Regional Development Fund; by the Ministry for Digital Transformation and Public Function through grant USECHIP (TSI-069100-2023-001) of PERTE Chip Chair Program, funded by European Union – Next Generation EU; and by project SIMMRE, “Plan Complementario de I+D+i en el área de Biodiversidad (PCBIO)” funded by the European Union within the framework of the Recovery, Transformation and Resilience Plan - NextGenerationEU, by the Spanish Ministry of Science, Innovation and Universities and by the Regional Government of Andalucía.

References

- Abadi, M., et al., 2016. TensorFlow: a system for large-scale machine learning. In: 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16), pp. 265–283.
- Beery, S., Van Horn, G., Perona, P., 2018. Recognition in terra incognita. In: Proceedings of the European Conference on Computer Vision (ECCV).
- Ackerman, S., Farchi, E., Raz, O., Zalmanovici, M., Dube, P., 2020. Detection of data drift and outliers affecting machine learning model performance over time. In JSM Proceedings, Nonparametric Statistics Section. Am. Stat. Assoc. 144–160.
- Beery, S., Morris, D., Yang, S., 2019. Efficient pipeline for camera trap image review. arXiv, 1907.06772.
- Benoit-Cattin, T., Velasco-Montero, D., Fernández-Berni, J., 2020. Impact of thermal throttling on long-term visual inference in a CPU-based edge device. Electronics 9 (12). <https://doi.org/10.3390/electronics9122106>. URL: <https://www.mdpi.com/2079-9292/9/12/2106>.
- Brust, C., Burghardt, T., Groenenberg, M., Kading, C., Kühl, H.S., Manguette, M.L., Denzler, J., 2017. Towards automated visual monitoring of individual gorillas in the wild. In: 2017 IEEE International Conference on Computer Vision Workshops (ICCVW), pp. 2820–2830. <https://doi.org/10.1109/ICCVW.2017.333>.
- Chen, G., Han, T.X., He, Z., Kays, R., Forrester, T., 2014. Deep convolutional neural network based species recognition for wild animal monitoring. In: 2014 IEEE International Conference on Image Processing (ICIP), pp. 858–862.
- Chen, R., Little, R., Mihaylova, L., Delahay, R., Cox, R., 2019. Wildlife surveillance using deep learning methods. Ecol. Evol. 9 (17), 9453–9466. <https://doi.org/10.1002/ece3.5410>.
- Christin, S., Hervet, E., Lecomte, N., 2019. Applications for deep learning in ecology. Methods in Ecology and Evolution 10. <https://doi.org/10.1111/2041-210X.13256>.
- Cossu, A., Ziosi, M., Lomonaco, V., 2021. Sustainable artificial intelligence through continual learning. Proceedings of the 1st International Conference on AI for People: Towards Sustainable AI, CAIP, 20–24. Bologna, Italy. <https://doi.org/10.4108/eai.20-11-2021.2314097>.
- Cunha, F., dos Santos, E.M., Barreto, R.S., Colonna, J.G., 2021. Filtering empty camera trap images in embedded systems, 2104.08859. URL: <https://arxiv.org/abs/2104.08859>.
- Curtin, B.H., Matthews, S.J., 2019. Deep learning for inexpensive image classification of wildlife on the Raspberry Pi. 2019 IEEE 10th Annual Ubiquitous Computing, Electronics Mobile Communication Conference (UEMCON), pp. 0082–0087.
- De Lange, M., Aljundi, R., Masana, M., Parisot, S., Jia, X., Leonardi, A., Slabaugh, G., Tuytelaars, T., 2022. A continual learning survey: defying forgetting in classification tasks. IEEE Trans. Pattern Anal. Mach. Intell. 44 (7), 3366–3385. <https://doi.org/10.1109/TPAMI.2021.3057446>.
- Deb, D., Wiper, S., Gong, S., Shi, Y., Tymoszek, C., Fletcher, A., Jain, A.K., 2018. Face recognition: primates in the wild. In: 2018 IEEE 9th International Conference on Biometrics Theory, Applications and Systems (BTAS), pp. 1–10. <https://doi.org/10.1109/BTAS.2018.8698538>.
- DeepFaune, 2024. <https://www.deepfaune.cnrs.fr/en/>.
- Dertien, J.S., Negi, H., Dinerstein, E., Krishnamurthy, R., Negi, H.S., Gopal, R., Gulick, S., Pathak, S.K., Kapoor, M., Yadav, P., Benitez, M., Ferreira, M., Wijnveen, A.J., Lee, A. T.L., Wright, B., Baldwin, R.F., 2023. Mitigating human–wildlife conflict and monitoring endangered tigers using a real-time camera-based alert system. BioScience 73 (10), 748–757. <https://doi.org/10.1093/biosci/biad076>.
- Doñana Biological Station, 2024. Conservation Biology and Global Change Research Group. <http://www.ebd.csic.es/biologia-de-la-conservacion-y-cambio-global>.
- Elias, A.R., Golubovic, N., Krintz, C., Wolski, R., 2017. Where's the bear? - automating wildlife image processing using IoT and edge cloud systems. In: 2017 IEEE/ACM Second International Conference on Internet-of-Things Design and Implementation (IoTDI), pp. 247–258.
- French, R.M., 1999. Catastrophic forgetting in connectionist networks. Trends Cogn. Sci. 3 (4), 128–135. [https://doi.org/10.1016/S1364-6613\(99\)01294-2](https://doi.org/10.1016/S1364-6613(99)01294-2). URL: <https://www.sciencedirect.com/science/article/pii/S1364661399012942>.
- Freytag, A., Rodner, E., Simon, M., Loos, A., Kuhl, H., Denzler, J., 2016. Chimpanzee Faces in the Wild: Log-Euclidean CNNs for Predicting Identities and Attributes of Primates, 9796, pp. 51–63.
- GitHub Microsoft, 2024. CameraTraps Repository. <https://github.com/microsoft/CameraTraps/>.
- Gomez Villa, A., Salazar, A., Vargas, F., 2017. Towards automatic wild animal monitoring: identification of animal species in camera-trap images using very deep convolutional neural networks. Eco. Inform. 41, 24–32. <https://doi.org/10.1016/j.ecoinf.2017.07.004>. URL: <http://www.sciencedirect.com/science/article/pii/S1574954116302047>.
- Gomez, A., Diez, G., Salazar, A., Diaz, A., 2016. Animal identification in low quality camera-trap images using very deep convolutional neural networks and confidence thresholds. In: Bebis, G., Boyle, R., Parvin, B., Koracin, D., Porikli, F., Skaff, S., Entezari, A., Min, J., Iwai, D., Sadagic, A., Scheidegger, C., Isenberg, T. (Eds.), Advances in Visual Computing. Springer International Publishing, Cham, pp. 747–756.
- Guo, Y., Rothfus, T., Ashour, A.S., Si, L., Chunlai, D., Ting, T.-F., 2019. A varied channels region proposal and classification network for wildlife image classification under complex environment. IET Image Process. 14. <https://doi.org/10.1049/iet-ipr.2019.1042>.
- Huang, Y.-P., Basanta, H., 2019. Bird image retrieval and recognition using a deep learning platform. IEEE Access 7, 66980–66989. <https://doi.org/10.1109/ACCESS.2019.2918274>.
- Huettmann, F., Arhonditsis, G., 2023. Towards an ecological informatics scholarship that is reflective, repeatable, transparent, and sharable! Eco. Inform. 76, 102132. <https://doi.org/10.1016/j.ecoinf.2023.102132>.
- Iandola, F., Han, S., Moskewicz, M., Ashraf, K., Dally, W., Keutzer, K., 2016. SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and <1MB model size. arXiv, 1602.07360.
- Jolles, J.W., 2021. Broad-scale applications of the Raspberry Pi: a review and guide for biologists. Methods Ecol. Evol. 12 (9), 1562–1579. <https://doi.org/10.1111/2041-210X.13652>.
- Körshchens, M., Barz, B., Denzler, J., 2018. Towards automatic identification of elephants in the wild. arXiv, 1812.04418.
- L. I. L. of Alexandria, 2024. Biology, C. L. BC, Snapshot Serengeti. <http://lila.science/datasets/snapshot-serengeti>.
- Labeled Information Library of Alexandria, 2024. Biology and Conservation (LILA BC), Camera Traps Datasets. <http://lila.science/category/camera-traps/>.
- Labeled Information Library of Alexandria: Biology and Conservation (LILA BC), 2024. Caltech Camera Traps. <http://lila.science/datasets/caltech-camera-traps>.
- Labeled Information Library of Alexandria: Biology and Conservation (LILA BC), 2024. North American Camera Trap Images. <http://lila.science/datasets/nacti>.
- Lesort, T., Lomonaco, V., Stoian, A., Maltoni, D., Filliat, D., Díaz-Rodríguez, N., 2020. Continual learning for robotics: definition, framework, learning strategies, opportunities and challenges. Information Fusion 58, 52–68. <https://doi.org/10.1016/j.inffus.2019.12.004>. URL: <https://www.sciencedirect.com/science/article/pii/S1566253519307377>.
- Loos, A., Weigel, C., Koehler, M., 2018. Towards automatic detection of animals in camera-trap images. In: 2018 26th European Signal Processing Conference (EUSIPCO), pp. 1805–1809. <https://doi.org/10.23919/EUSIPCO.2018.8553439>.
- Maltoni, D., Lomonaco, V., 2019. Continuous learning in single-incremental-task scenarios. Neural Networks 116, 56–73. <https://doi.org/10.1016/j.neunet.2019.03.010>.

- Mathur, A., Khattar, S., June 2019. Real-Time Wildlife Detection on Embedded Systems. Technical report. Stanford University. URL: <http://ilpubs.stanford.edu:8090/1165/>.
- Miao, Z., Gaynor, K.M., Wang, J., Liu, Z., Muellerklein, O., Norouzzadeh, M.S., McInturff, A., Bowie, R.C.K., Nathan, R., Yu, S.X., Getz, W.M., 2019. Insights and approaches using deep learning to classify wildlife. *Sci. Rep.* 9 (1), 8137. <https://doi.org/10.1038/s41598-019-44565-w>. URL: <https://europepmc.org/articles/P/MC6544615>.
- Miguel, A., Beery, S., Flores, E., Klemesrud, L., Bayrakcismith, R., 2016. Finding areas of motion in camera trap images. In: 2016 IEEE International Conference on Image Processing (ICIP), pp. 1334–1338. <https://doi.org/10.1109/ICIP.2016.7532575>.
- Nguyen, H., MacLagan, S.J., Nguyen, T.D., Nguyen, T., Flemons, P., Andrews, K., Ritchie, E.G., Phung, D., 2017. Animal recognition and identification with deep convolutional neural networks for automated wildlife monitoring. In: 2017 IEEE International Conference on Data Science and Advanced Analytics (DSAA), pp. 40–49. <https://doi.org/10.1109/DSAA.2017.31>.
- Norouzzadeh, M.S., Nguyen, A., Kosmala, M., Swanson, A., Palmer, M.S., Packer, C., Clune, J., 2018. Automatically identifying, counting, and describing wild animals in camera-trap images with deep learning. *Proceedings of the National Academy of Sciences* 115 (25), E5716–E5725. <https://doi.org/10.1073/pnas.1719367115>. URL: <https://www.pnas.org/content/115/25/E5716>.
- OpenCV, 2024. <https://opencv.org/>.
- Panasonic, 2024. PIR Motion Sensor PaPIRs. <https://www3.panasonic.biz/ac/e/control/sensor/human/index.jsp/>.
- Parham, J., Stewart, C., Crall, J., Rubenstein, D., Holmberg, J., Berger-Wolf, T., 2018. An animal detection pipeline for identification. In: 2018 IEEE Winter Conference on Applications of Computer Vision (WACV), pp. 1075–1083. <https://doi.org/10.1109/WACV.2018.00123>.
- Pellegrini, L., Graffieti, G., Lomonaco, V., Maltoni, D., 2020. Latent replay for real-time continual learning. In: 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 10203–10209. <https://doi.org/10.1109/IROS45743.2020.9341460>.
- Pigpio Library, 2024. <http://abyz.me.uk/rpi/pigpio/download.html>.
- Raspberry Pi, 2024. Operating System Images. <https://www.raspberrypi.com/software/operating-systems/>.
- Raspberry Pi 4, 2024. <https://www.raspberrypi.org/products/raspberry-pi-4-model-b/>.
- Raspberry Pi Documentation, 2024. Accessories. Camera. <https://www.raspberrypi.org/documentation/accessories/camera.html>.
- Raspberry Pi Zero 2W, 2024. <https://www.raspberrypi.com/products/raspberry-pi-zero-2-w/>.
- Rigoudy, N., Dussert, G., Benyoub, A., Besnard, A., Birck, C., Boyer, J., Bollet, Y., Bunz, Y., Caussimont, G., Chetouane, E., Carriburu, J.C., Cornette, P., Delestrade, A., Backer, N.D., Dispan, L., Barh, M.L., Duhayer, J., Elder, J.-F., Fanjul, J.-B., Fonderlick, J., Froustey, N., Garel, M., Gaudry, W., Gérard, A., Gimenez, O., Hemery, A., Hemon, A., Jullien, J.-M., Knitter, D., Malafosse, I., Marginean, M., Ménard, L., Ouvrier, A., Pariset, G., Prunet, V., Rabault, J., Randon, M., Raulet, Y., Régnier, A., Ribière, R., Ricci, J.-C., Ruette, S., Schneynlin, Y., Sentilles, J., Siefert, N., Smith, B., Terpereau, G., Touchet, P., Thuiller, W., Uzal, A., Vautrain, V., Vimal, R., Weber, J., Spataro, B., Miele, V., Chamailé-Jammes, S., 2023. The DeepFaune initiative: a collaborative effort towards the automatic identification of french fauna in camera-trap images. *Eur. J. Wildl. Res.* 69, 113. <https://doi.org/10.1007/s10344-023-01742-7>.
- Rodríguez, N.D., Lomonaco, V., Filliat, D., Maltoni, D., 2018. Don't forget, there is more than forgetting: new metrics for continual learning. *arXiv arXiv:1810.13166*. URL: <http://arxiv.org/abs/1810.13166>.
- Russakovsky, O., Deng, J., Su, H., Krause, J., Satheesh, S., Ma, S., Huang, Z., Karpathy, A., Khosla, A., Bernstein, M., Berg, A., Fei-Fei, L., 2015. ImageNet large scale visual recognition challenge. *International Journal of Computer Vision (IJCV)* 115 (3), 211–252. <https://doi.org/10.1007/s11263-015-0816-y>.
- S. C.K. S. E., 2019. Automated wildlife monitoring using deep learning. In: *Proceedings of the International Conference on Systems, Energy & Environment (ICSEE)*, 2019.
- Schneider, S., Taylor, G.W., Kremer, S.C., 2018a. Deep learning object detection methods for ecological camera trap data. In: 2018 15th Conference on Computer and Robot Vision (CRV), pp. 321–328.
- Schneider, S., Taylor, G., Linquist, S., Kremer, S., 2018b. Past, present, and future approaches using computer vision for animal re-identification from camera trap data. *Methods in Ecology and Evolution* 10. <https://doi.org/10.1111/2041-210X.13133>.
- Schneider, S., Greenberg, S., Taylor, G.W., Kremer, S.C., 2020a. Three critical factors affecting automated image species recognition performance for camera traps. *Ecol. Evol.* 10 (7), 3503–3517. <https://doi.org/10.1002/ece3.6147>.
- Schneider, S., Taylor, G.W., Kremer, S.C., 2020b. Similarity learning networks for animal individual re-identification - beyond the capabilities of a human observer. In: 2020 IEEE Winter Applications of Computer Vision Workshops (WACVW), pp. 44–52. <https://doi.org/10.1109/WACVW50321.2020.9096925>.
- Selvaraju, R.R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., Batra, D., 2017. Grad-CAM: visual explanations from deep networks via gradient-based localization. In: 2017 IEEE International Conference on Computer Vision (ICCV), pp. 618–626. <https://doi.org/10.1109/ICCV.2017.74>.
- Sharma, K., Fiechter, M., George, T., Young, J., Alexander, J.S., Bijoor, A., Suryawanshi, K., Mishra, C., 2020. Conservation and people: towards an ethical code of conduct for the use of camera traps in wildlife research. *Ecological Solutions and Evidence* 1 (2), e12033. <https://doi.org/10.1002/2688-8319.12033>.
- Singh, A., Pietrasik, M., Natha, G., Ghouaiel, N., Brizel, K., Ray, N., 2020. Animal detection in man-made environments. In: 2020 IEEE Winter Conference on Applications of Computer Vision (WACV), pp. 1427–1438. <https://doi.org/10.1109/WACV45572.2020.9093504>.
- Systema Naturae, 2023. Conservation Wildlife Datasets. <https://www.systemanaturae.org/>.
- Tabak, M.A., Norouzzadeh, M.S., Wolfson, D.W., Sweeney, S.J., Vercauteren, K.C., Snow, N.P., Halseth, J.M., Di Salvo, P.A., Lewis, J.S., White, M.D., Teton, B., Beasley, J.C., Schlichting, P.E., Boughton, R.K., Wight, B., Newkirk, E.S., Ivan, J.S., Odell, E.A., Brook, R.K., Lukacs, P.M., Moeller, A.K., Mandeville, E.G., Clune, J., Miller, R.S., 2019. Machine learning to classify animal species in camera trap images: applications in ecology. *Methods in Ecology and Evolution* 10 (4), 585–590. <https://doi.org/10.1111/2041-210X.13120>.
- Tan, T.F., Teoh, S.S., Fow, J.E., Yen, K.S., 2016. Embedded human detection system based on thermal and infrared sensors for anti-poaching application. In: 2016 IEEE Conference on Systems, Process and Control (ICSPC), pp. 37–42. <https://doi.org/10.1109/SPC.2016.7920700>.
- Velasco-Montero, D., 2024. Research on Smart Camera Traps Repository. https://github.com/DVM000/smart_camera_trap_research/.
- Velasco-Montero, D., Sanglas, A., 2024. Dataset I (feral cat, other, blank). Dataset II collected by camera trap prototype (animal, blank). <https://zenodo.org/records/10687510>.
- Velasco-Montero, D., Fernández-Berni, J., Carmona-Galán, R., Rodríguez-Vázquez, A., 2018. Optimum selection of DNN model and framework for edge inference. *IEEE Access* 6, 51680–51692. <https://doi.org/10.1109/ACCESS.2018.2869929>.
- Velasco-Montero, D., Fernández-Berni, J., Rodríguez-Vázquez, A., 2022. Visual Inference for IoT Systems: A Practical Approach. Springer International Publishing. <https://doi.org/10.1007/978-3-030-90903-1>.
- Wang, H., Song, Y., Li, S., Dai, W., Liu, J., 2019. Transfer learning based wildlife recognition for tele-observation in field occlusion environment. In: 2019 IEEE International Conference on Image Processing (ICIP), pp. 3392–3396. <https://doi.org/10.1109/ICIP.2019.8803691>.
- Ward, S., Hensler, J., Alsalam, B., Gonzalez, L.F., 2016. Autonomous UAVs wildlife detection using thermal imaging, predictive navigation and computer vision. In: 2016 IEEE Aerospace Conference, pp. 1–8. <https://doi.org/10.1109/AERO.2016.7500671>.
- Weinstein, B.G., 2018. Scene-specific convolutional neural networks for video-based biodiversity detection. *Methods in Ecology and Evolution* 9 (6), 1435–1441. <https://doi.org/10.1111/2041-210X.13011>.
- Whytock, R.C., Świeżewski, J., Zwerts, J.A., Bara-Stupski, T., Koumba Pambo, A.F., Rogala, M., Din, L. Bahaa-El, Boekke, K., Brittain, S., Cardoso, A.W., Henschel, P., Lehmann, D., Momboua, B., Opepa, C., Kiebo, Orbell, C., Pitman, R.T., Robinson, H. S., Abernethy, K.A., 2021. Robust ecological analysis of camera trap data labelled by a machine learning model. *Methods Ecol. Evol.* 12 (6), 1080–1092. <https://doi.org/10.1111/2041-210X.13576>.
- Whytock, R.C., Suijten, T., van Deursen, T., Świeżewski, J., Meriaghie, H., Madamba, N., Mouckoumou, N., Zwerts, J.A., Pambo, A.F.K., Din, L. Bahaa-El, Brittain, S., Cardoso, A.W., Henschel, P., Lehmann, D., Momboua, B.R., Makaga, L., Orbell, C., White, L.J.T., Iponga, D.M., Abernethy, K.A., 2023. Real-time alerts from AI-enabled camera traps using the Iridium satellite network: a case-study in Gabon, Central Africa. *Methods Ecol. Evol.* 14 (3), 867–874. <https://doi.org/10.1111/2041-210X.14036>.
- Willi, M., Pitman, R.T., Cardoso, A.W., Locke, C., Swanson, A., Boyer, A., Veldhuis, M., Fortson, L., 2019. Identifying animal species in camera trap images using deep learning and citizen science. *Methods Ecol. Evol.* 10 (1), 80–91. <https://doi.org/10.1111/2041-210X.13099>.
- Willi, M., Pitman, R.T., Cardoso, A.W., Locke, C., Swanson, A., Boyer, A., Veldhuis, M., Fortson, L., 2024. Software, Data & Models used in "Identifying Animal Species in Camera Trap Images using Deep Learning and Citizen Science". <https://conservancy.umn.edu/handle/11299/199818>.
- Xie, N., Ras, G., van Gerven, M., Doran, D., 2022. Explainable deep learning: a field guide for the uninitiated. *J. Artif. Intell. Res.* 73. <https://doi.org/10.1613/jair.1.13200>.
- Yousif, H., Yuan, J., Kays, R., He, Z., 2017. Fast human-animal detection from highly cluttered camera-trap images using joint background modeling and deep learning classification. In: 2017 IEEE International Symposium on Circuits and Systems (ISCAS), pp. 1–4. <https://doi.org/10.1109/ISCAS.2017.8050762>.
- Yousif, H., Yuan, J., Kays, R., He, Z., 2019. Animal scanner: software for classifying humans, animals, and empty frames in camera trap images. *Ecol. Evol.* 9 (4), 1578–1589. <https://doi.org/10.1002/ece3.4747>.
- Zhang, Z., He, Z., Cao, G., Cao, W., 2016. Animal detection from highly cluttered natural scenes using spatiotemporal object region proposals and patch verification. *IEEE Trans. Multimed.* 18 (10), 2079–2092. <https://doi.org/10.1109/TMM.2016.2594138>.
- Zhuang, F., Qi, Z., Duan, K., Xi, D., Zhu, Y., Zhu, H., Xiong, H., He, Q., 2021. A comprehensive survey on transfer learning. *Proc. IEEE* 109 (1), 43–76. <https://doi.org/10.1109/JPROC.2020.3004555>.
- Zualkernan, I.A., Dhous, S., Judas, J., Sajun, A., Gomez, B.R., Hussain, L.A., 2022. An IoT system using deep learning to classify camera trap images on the edge. *Computers* 11 (1), 13. <https://doi.org/10.3390/computers11010013>.